

UNIT-IV

STORAGE MANAGEMENT

MASS STORAGE STRUCTURE

1. Overview of Mass Storage Structure

Magnetic Disks

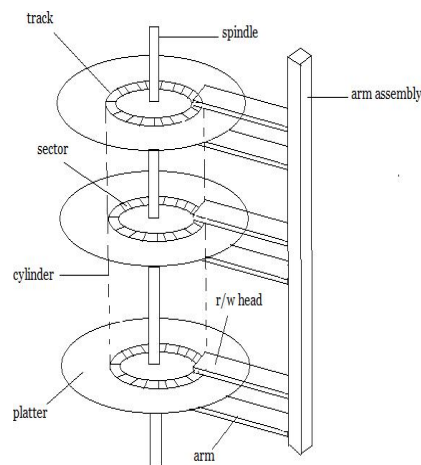
- ❖ In modern computers, most of the secondary storage is in the form of magnetic disks.
- ❖ A magnetic disk contains several platters. Each platter is divided into circular shaped tracks.
- ❖ The length of the tracks near the centre is less than the length of the tracks farther from the centre.
- ❖ Each track is further divided into sectors.
- ❖ Tracks of the same distance from centre form a cylinder.
- ❖ A read-write head is used to read data from a sector of the magnetic disk.
- ❖ The speed of the disk is measured as two parts:

Transfer rate: This is the rate at which the data moves from disk to the computer.

Random access time: It is the sum of the seek time and rotational latency.

Seek time is the time taken by the arm to move to the required track.

Rotational latency is defined as the time taken by the arm to reach the required sector in the track.



Solid-State Disks

SD is non-volatile memory that is used like a hard drive. SSDs have the same characteristics as traditional hard disks but can be more reliable because they have no moving parts and faster because they have no seek time or latency. In addition, they consume less power.

SSDs have less capacity than the larger hard disks, and may have shorter life spans. use for SSDs is in storage arrays, where they hold file- system metadata that require high performance. Some SSDs are designed to connect directly to the system bus.

Magnetic Tapes

Magnetic tape was used as an early secondary-storage medium. Although it is relatively permanent and can hold large quantities of data, its access time is slow compared with that of main memory and magnetic disk.

In addition, random access to magnetic tape is about a thousand times slower than random access to magnetic disk, so tapes are not very useful for secondary storage.

2. Disk Structure

In Disk drives are addressed as large 1-dimensional arrays of logical blocks, where the logical block is the smallest unit of transfer. n The 1-dimensional array of logical blocks is mapped into the sectors of the disk sequentially.

→ In Sector 0 is the first sector of the first track on the outermost cylinder.

→ In Mapping proceeds in order through that track, then the rest of the tracks in that cylinder, and then through the rest of the cylinders from outermost to innermost.

3. Disk Scheduling

The operating system is responsible for using hardware efficiently.

For the disk drives, this means having a fast access time & disk bandwidth.

→ Access time has two major components:

→ Seek time is the time for the disk to move the heads to the cylinder containing the desired sector.

→ Rotational latency time waiting for the disk to rotate the desired sector to the disk head.

→ We like to minimize seek time.

→ Disk bandwidth is the total number of bytes transferred divided by the total time between the first request for service and the completion of the last transfer.

→ Several algorithms exist to schedule the servicing of disk I/O requests.

We illustrate them with a Request Queue (cylinder range 0-199):

98, 183, 37, 122, 14, 124, 65, 67

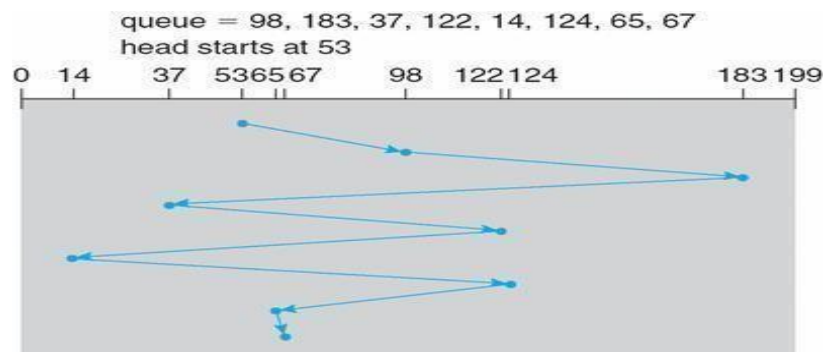
Head pointer: cylinder 53

1. First Come First Serve

This algorithm performs requests in the same order asked by the system. Let's take an example where the queue has the following requests with cylinder numbers as follows:

98, 183, 37, 122, 14, 124, 65, 67

Illustration shows total head movement of 640 cylinders

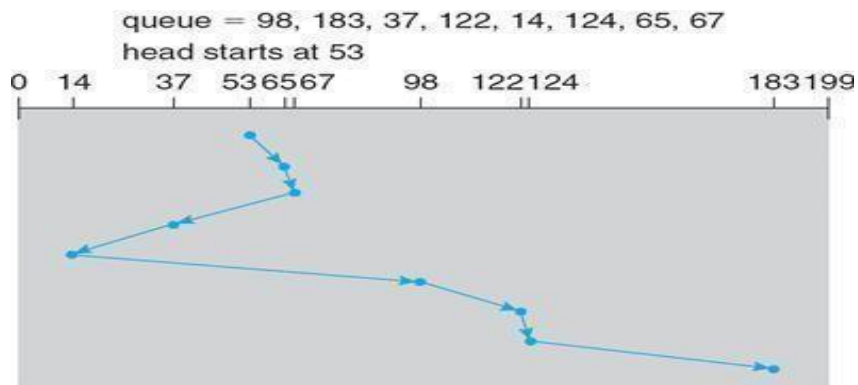


2. SSTF (Shortest Seek Time First)

Selects the request with the minimum seek time from the current head position.

SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.

Illustration shows total head movement of 236 cylinders.



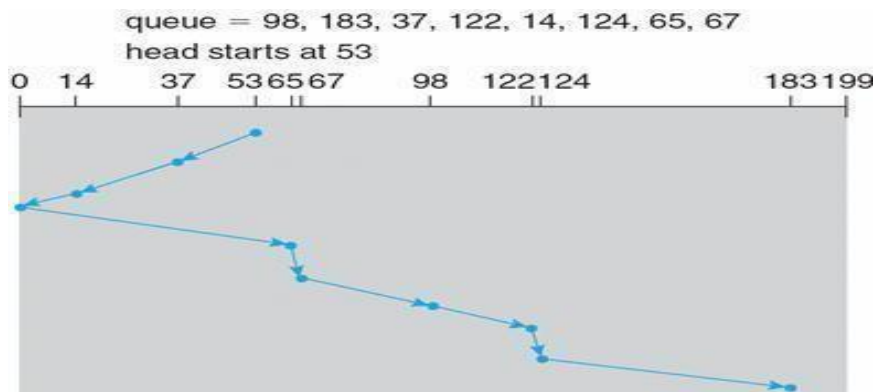
3. SCAN

The disk arm starts at one end of the disk, and moves toward the other end, servicing requests

until it gets to the other end of the disk, where the head movement is reversed and servicing continues.

SCAN algorithm sometimes called the elevator algorithm.

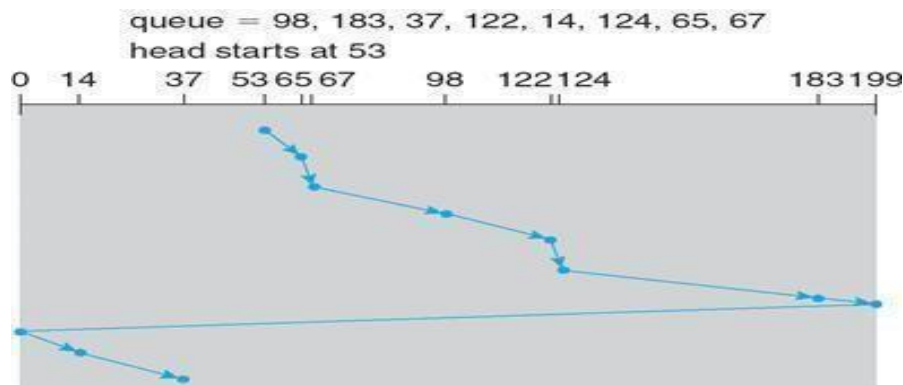
Illustration shows total head movement of 208 cylinders



4. C-SCAN

Provides a more uniform wait time than SCAN. The head moves from one end of the disk to the other, servicing requests as it goes. When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.

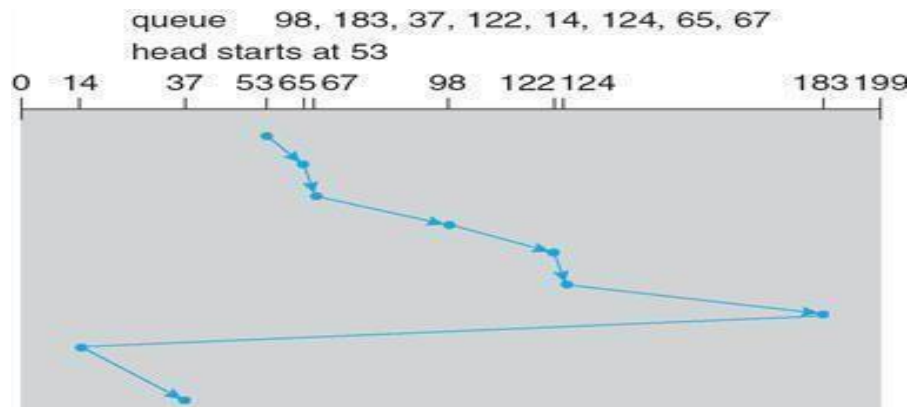
Treats the cylinders as a Circular list that wraps around from the last cylinder to the first one.



5. LOOK

→ Version of C-SCAN

→ Arm only goes as far as the **last request** in each direction, then reverses direction immediately, without first going all the way to the end of the disk.



4. Disk Management

The operating system is responsible for disk initialization, booting from disk, and bad-block recovery.

Disk Formatting

A new magnetic disk must be divided into sectors that the disk controller can read and write. This process is called **low-level formatting, or physical formatting**. Low-level formatting fills the disk with a special data structure for each sector. The data structure for a sector typically consists of a header, a data area (usually 512 bytes in size), and a trailer.

The header and trailer contain information used by the disk controller, such as a sector number and an **error-correcting code (ECC)**.

This formatting enables the manufacturer to 1. Test the disk and 2. To initialize the mapping from logical block numbers

To use a disk to hold files, the operating system still needs to record its own data structures on the disk.

It does so in two steps.

(a) The first step is **Partition** the disk into one or more groups of cylinders.

Among the partitions, one partition can hold a copy of the OS's executable code, while another holds user files.

(b)The second step is **logical formatting**. The operating system stores the initial file-system data structures onto the disk. These data structures may include maps of free and allocated space and an initial empty directory.

Boot Block

For a computer to start running-for instance, when it is powered up or rebooted-it needs to have an initial program to run. This initial program is called bootstrap program & it should be simple.

It initializes all aspects of the system, from CPU registers to device controllers and the contents of main memory, and then starts the operating system.

The bootstrap is stored in read-only memory (ROM). This location is convenient, because ROM needs no initialization and is at a fixed location that the processor can start executing when powered up or reset. And, since ROM is read only, it cannot be infected by a computer virus.

The full bootstrap program is stored in the “**boot blocks**” at a fixed location on the disk. A disk that has a boot partition is called a boot disk or system disk. The work of boot block as follows

1. Finds the operating system kernel on disk,
2. Loads that kernel into memory, and
3. Jumps to an initial address to begin the operating-system execution.

The full **bootstrap program** is stored in a partition called the boot blocks, at a fixed location on the disk. A disk that has a boot partition is called a boot disk or system disk.

The code in the boot ROM instructs the disk controller to read the boot blocks into memory and then starts executing that code.

Bootstrap loader - load the entire operating system from a non-fixed location

on disk, and to start the operating system running.

Bad Blocks

The disk with defected sector is called as bad block. Depending on the disk and controller in use, these blocks are handled in a variety of ways;

Method 1: “Handled manually”

If blocks go bad during normal operation, a **special program** must be run manually to search for the bad blocks and to lock them away as before. Data that resided on the bad blocks usually are lost.

Method 2: “sector sparing or forwarding”

The controller maintains a list of bad blocks on the disk. Then the controller can be told to replace each bad sector logically with one of the spare sectors. This scheme is known as sector sparing or forwarding.

A typical bad-sector transaction might be as follows:

- The operating system tries to read logical block 87.
- The controller calculates the ECC and finds that the sector is bad.
- It reports this finding to the operating system.
- The next time that the system is rebooted, a special command is run to tell the controller to replace the bad sector with a spare.
- After that, whenever the system requests logical block 87, the request is translated into the replacement sector's address by the controller.

Method 3: “sector slipping”

For an example, suppose that logical block 17 becomes defective, and the first available spare follows sector 202. Then, sector slipping would remap all the sectors from 17 to 202, moving them all down one spot. That is, sector 202 would be copied into the spare, then sector 201 into 202, and then 200 into 201, and so on, until sector 18 is copied into sector 19. Slipping the sectors in this way frees up the space of sector 18, so sector 17 can be mapped to it.

5. Swap-Space Management

- ➔ Swap-space — virtual memory uses disk space as an extension of main memory.
- ➔ Main goal for the design and implementation of swap space is to provide the best throughput for VM system

1. Swap-space use

Swapping –use swap space to hold entire process image

Paging –store pages that have been pushed out of memory

→ Some OS may support multiple swap-space

–Put on separate disks to balance the load

→ Better to overestimate than underestimate

–If out of swap-space, some processes must be aborted or system crashed

2. Swap-Space Location

→ Swap-space can be carved out of the normal file system, or in a separate disk partition

→ A large file within the file system: simple but inefficient

–Navigating the directory structure and the disk-allocation data structure takes time and potentially extra disk accesses

–External fragmentation can greatly increase swapping times by forcing multiple seeks during reading or writing of a process image

–Improvement

Caching block location information in main memory

•Contiguous allocation for the swap file

But, the cost of traversing FS data structure still remains

→ In a separate partition: raw partition

–Create a swap space during disk partitioning

–A separate swap-space storage manager is used to allocate and de-allocate blocks

–Use algorithms optimized for speed, rather than storage efficiency

–Internal fragment may increase

Linux supports both approaches

→ Swap-space Management: Example

Solaris 1

–Text-segment pages are brought in from the file system and are thrown away if selected for paged out

More efficient to re-read from FS than write it to the swap space

–Swap space: only used as a backing store for pages of anonymous

memory

Stack, heap, and uninitialized data

Solaris 2

–Allocates swap space only when a page is forced out of physical memory

Not when the virtual memory page is first created.

FILE SYSTEM INTERFACE

1. File Concepts

A file is a named collection of related information that is recorded on secondary storage. From user's perspective a , a file is the smallest allotment of that logical secondary storage; unless they are within a file.

Commonly, files represent programs (both source and object forms) and data. Data files may be numeric, alphabetic, alphanumeric, or binary.

In general, a file is a sequence of bits, bytes, lines, or records, the meaning of which is defined by the file's creator and user.

A **text file** is a sequence of characters organized into lines (and possibly pages). An **executable** file is a series of code sections that the loader can bring into memory and execute.

2. File Attributes

The information about all files is kept in the directory structure, a directory entry consists of the file's name and its unique identifier. The identifier in turn locates the other file attributes.

- **Name:** The symbolic file name is the only information kept in human readable form.
- **Identifier:** This unique tag, usually a number identifies the file within the file system. It is the non-human readable name for the file.
- **Type:** This information is needed for those systems that support different types.
- **Location:** This information is a pointer to a device and to the location of the file on that device.
- **Size:** The current size of the file (in bytes, words or blocks) and possibly the maximum allowed size are included in this attribute.

- **Protection:** Access-control information determines who can do reading, writing, executing and so on.
- **Time, date and user identification:** This information may be kept for creation, last modification and last use. These data can be useful for protection, security and usage monitoring.

3. File Operations

The operating system can provide system calls to create, write, read, reposition, delete, and truncate files.

Creating a file - First, space in the file system must be found for the file, Second, an entry for the new file must be made in the directory.

Writing a file - System call specifying both the name of the file and the information to be written to the file.

Reading a file - we use a system call that specifies the name of the file and where (in memory) the next block of the file should be put.

Repositioning within a file - The directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value.

Deleting a file - search the directory for the named file. Having found the associated directory entry, we release all file space

Truncating a file - this function allows all attributes to remain unchanged—except for file length—but lets the file be reset to length zero and its file space released.

4. File Types

file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

5. Access Methods

1. Sequential Access

- ❖ Data is accessed one record right after another in an order.
- ❖ Read command causes a pointer to be moved ahead by one.
- ❖ Write command allocates space for the record and moves the pointer to the new End of File.
- ❖ Such a method is reasonable for tape.



2. Direct Access

- ❖ This method is useful for disks.
- ❖ The file is viewed as a numbered sequence of blocks or records.
- ❖ There are no restrictions on which blocks are read/written, it can be done in any order.

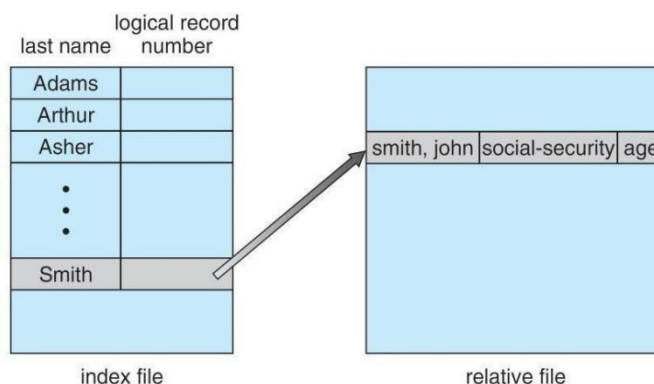
- ❖ User now says "read n" rather than "read next".
- ❖ "n" is a number relative to the beginning of file, not relative to an absolute physical disk location.

As a simple example, on an **airline – reservation system**, we might store all the information about a particular flight (for example, flight 713) in the block identified by the flight number.

Thus, the number of available seats for flight 713 is stored in block 713 of the reservation file. To store information about a larger set, such as people, we might compute a hash function on the people's names, or search a small in-memory index to determine a block to read and search.

3. Indexed Access

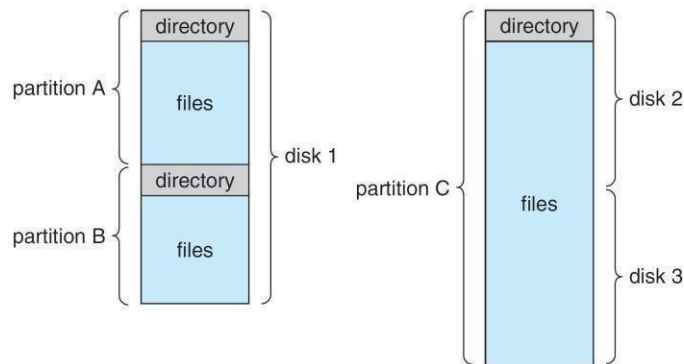
- ❖ If a file can be sorted on any of the filed then an index can be assigned to a group of certain records.
- ❖ However, A particular record can be accessed by its index.
- ❖ The index is nothing but the address of a record in the file.
- ❖ In index accessing, searching in a large database became very quick and easy but we need to have some extra space in the memory to store the index value.



6. Directory Structure

- ❖ Directory can be defined as the listing of the related files on the disk.
- ❖ The directory may store some or the entire file attributes.

- ❖ Each partition must have at least one directory in which, all the files of the partition can be listed.
- ❖ A directory entry is maintained for each file in the directory which stores all the information related to that file.



Operations that are to be performed on a directory

Search for a file. We need to be able to search a directory structure to find the entry for a particular file. Since files have symbolic names, and similar names may indicate a relationship among files, we may want to be able to find all files whose names match a particular pattern.

Create a file. New files need to be created and added to the directory.

Delete a file. When a file is no longer needed, we want to be able to remove it from the directory.

List a directory. We need to be able to list the files in a directory and the contents of the directory entry for each file in the list.

Rename a file. Because the name of a file represents its contents to its users, we must be able to change the name when the contents or use of the file changes. Renaming a file may also allow its position within the directory structure to be changed.

Traverse the file system. We may wish to access every directory and every file within a directory structure. For reliability, it is a good idea to save the contents and structure of the entire file system at regular intervals.

➔Often, we do this by copying all files to magnetic tape. This technique provides a backup copy in case of system failure.

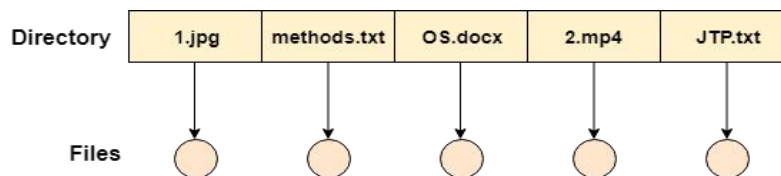
➔In addition, if a file is no longer in use, the file can be copied to tape and the disk space of that file released for reuse by another file.

Logical Structure (or) Level of Directory

- Single-level directory
- Two-level directory
- Tree-Structured directory
- Acyclic Graph directory
- General Graph directory

Single – Level Directory

- ❖ The simplest method is to have one big list of all the files on the disk.
- ❖ The entire system will contain only one directory which is supposed to mention all the files present in the file system.
- ❖ The directory contains one entry per each file present on the file system.



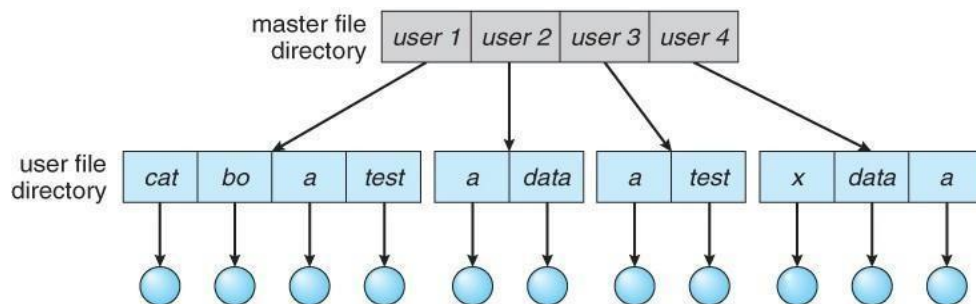
Single Level Directory

Disadvantages

1. We cannot have two files with the same name.
2. The directory may be very big therefore searching for a file may take so much time.
3. Protection cannot be implemented for multiple users.
4. There are no ways to group same kind of files.

Two Level Directory

- ❖ In two level directory systems, we can create a separate directory for each user.
- ❖ There is one master directory which contains separate directories dedicated to each user. For each user, there is a different directory present at the second level, containing group of user's file.
- ❖ The system doesn't let a user to enter in the other user's directory without permission.

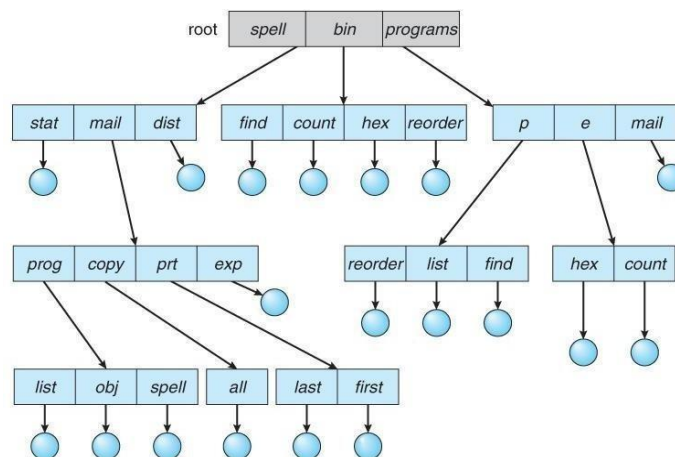


Characteristics of two level directory system

1. Each files has a path name as /User-name/directory-name/
2. Different users can have the same file name.
3. Searching becomes more efficient as only one user's list needs to be traversed.

Tree Structured Directory

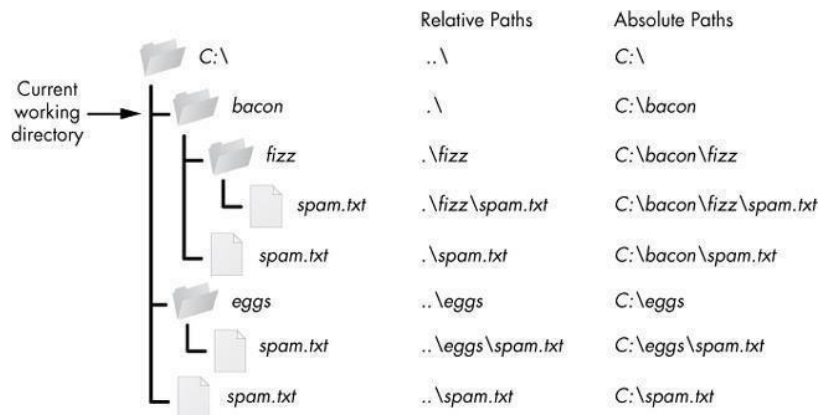
- ❖ Tree structured directory system overcomes the drawbacks of two level directory system.
- ❖ The similar kind of files can now be grouped in one directory.
- ❖ Each user has its own directory and it cannot enter in the other user's directory.



- ❖ Searching is more efficient in this directory structure

A file can be accessed by two types of path, either ➔ 1.Relative or 2. Absolute.

1. **Absolute path** is the path of the file with respect to the root directory of the system.
2. **Relative path** is the path with respect to the current working directory of the system



Acyclic-Graph Structured Directories

- ❖ When the **same files need to be accessed in more than one place** in the directory structure it can be useful to provide an acyclic-graph structure.
- ❖ In this system two or more directory entry can point to the same file or sub directory. That file or sub directory is shared between the two directory entries.

It provides two types of **links** for implementing the acyclic-graph structure

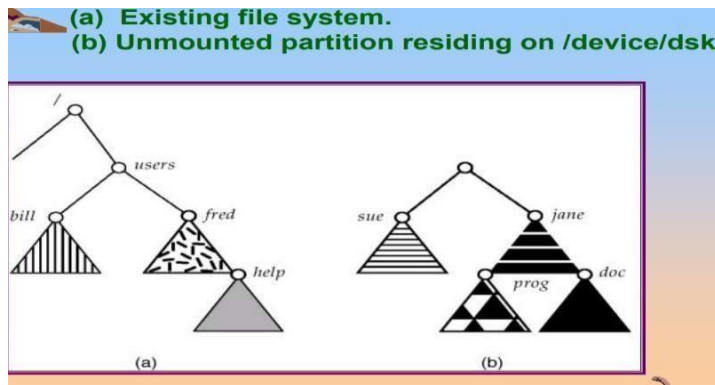
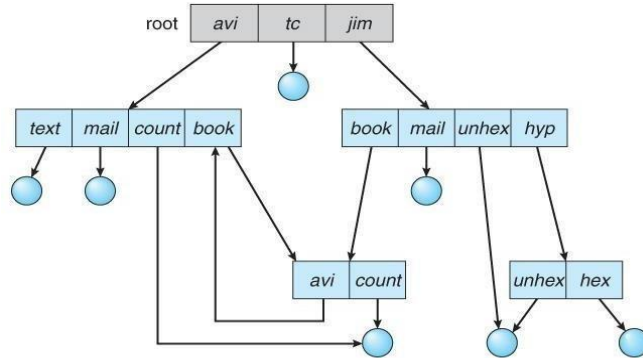
Soft link, the file just gets deleted and we are left with a dangling pointer.

Hard link, the actual file will be deleted only if all the references to it gets deleted.

General Graph Directory

- ❖ In general graph directory structure, cycles are allowed within a directory structure where multiple directories can be derived from more than one parent directory

- ❖ The main problem with this kind of directory structure is to calculate total size or

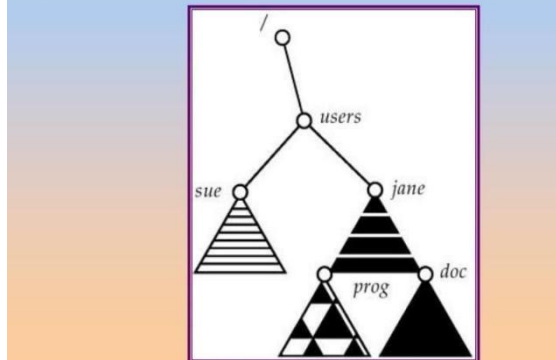


space that have been taken by the files and directories.

7. File System Mounting

- ❖ Before you can access the files on a file system, you need to mount the file system.
- ❖ Mounting a file system attaches that file system to a directory (mount point) and makes it available to the system.
- ❖ The root (/) file system is always mounted. Any other file system can be connected or disconnected from the root (/) file system.
- ❖ When you mount a file system, any files or directories in the underlying mount point directory are unavailable as long as the file system is mounted.
- ❖ These files are not permanently affected by the mounting process, and they become available again when the file system is unmounted.
- ❖ However, mount directories are typically empty, because you usually do not want to obscure existing files.

The effect of mounting partition over /users



8. File Sharing

- ❖ File sharing is the accessing or sharing of files by one or more users.
- ❖ File sharing is performed on computer networks as an easy and quick way to transmit data.

For example, a user may share an instruction document on his computer that is connected to a corporate network allowing all other employees to access and read that document.

1. Multiple Users

- ❖ On a multi-user system, more information needs to be stored for each file: The owner (user) who owns the file, and who can control its access.
- ❖ The group of other user IDs that may have some special access to the file.
- ❖ What access rights are afforded to the owner (User), the Group, and to the rest of the world.

2. Remote File Systems

The advent of the Internet introduces issues for accessing files stored on remote computers

- ❖ The original method was ftp, allowing individual files to be transported across systems as needed.
- ❖ The Client-Server Model (the system which physically owns the files acts as a *server*,

and the system which mounts them is the *client*.)

- ❖ Distributed Information Systems → service that runs on a single central location.

- ❖ Failure Modes → When a local disk file is unavailable, the result is generally known immediately, and is generally non-recoverable. The only reasonable response is for the response to fail. Remote access systems allow for blocking or delayed response.

3. Consistency Semantics

Consistency Semantics deals with the consistency between the views of shared files on a networked system. When one user changes the file, when do other users see the changes?

1. UNIX Semantics

- Writes to an open file are immediately visible to any other user who has the file open.

2. Session Semantics

AFS uses the following semantics:

- Writes to an open file are not immediately visible to other users.

- When a file is closed, any changes made become available only to users who open the file at a later time.

3. Immutable-Shared-Files Semantics

- when a file is declared as *shared* by its creator, it becomes immutable and the name cannot be re-used for any other resource. Hence it becomes read-only, and shared access is simple.

9. File Protection

- ❖ Files must be kept safe for reliability (against accidental damage), and protection(against deliberate malicious access.) The former is usually managed with backup copies. This section discusses the latter.
- ❖ One simple protection scheme is to remove all access to a file. However this makes the file unusable, so some sort of controlled access must be arranged.

Types of Access

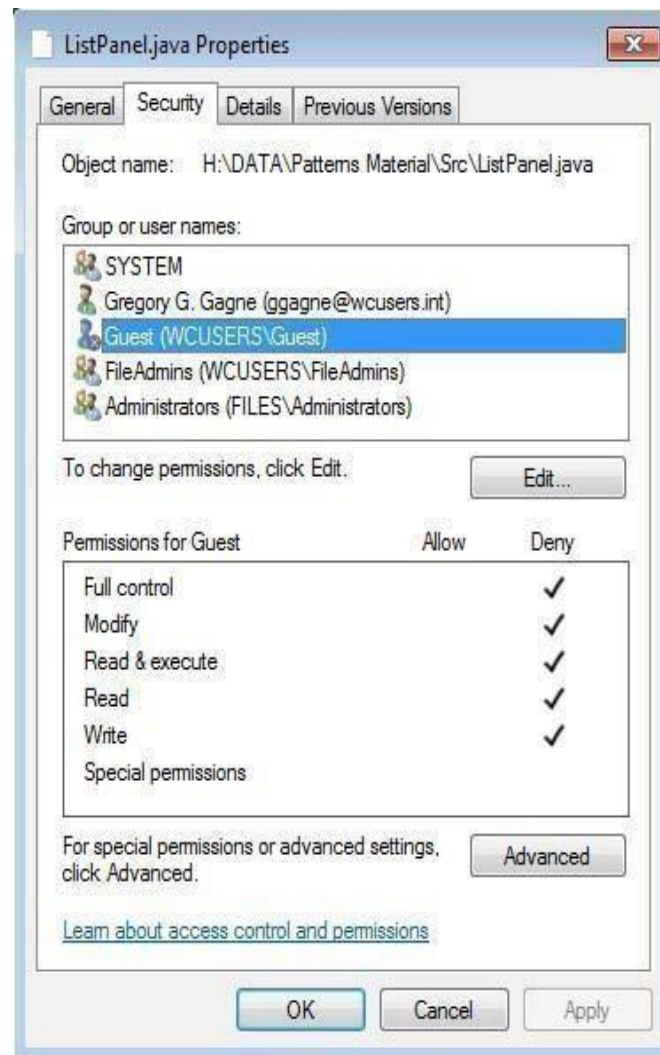
- The following low-level operations are often controlled:
 - Read - View the contents of the file
 - Write - Change the contents of the file.
 - Execute - Load the file onto the CPU and follow the instructions contained therein.
 - Append - Add to the end of an existing file.
 - Delete - Remove a file from the system.
 - List -View the name and other attributes of files on the system.
- Higher-level operations, such as copy, can generally be performed through combinations of the above.

Access Control

- One approach is to have complicated *Access Control Lists, ACL*, which specify exactly what access is allowed or denied for specific users or groups.
 - The AFS uses this system for distributed access.
 - Control is very finely adjustable, but may be complicated, particularly when the specific users involved are unknown. (AFS allows some wild cards, so for example all users on a certain remote system may be trusted, or a given username may be trusted when accessing from any remote system.)
- UNIX uses a set of 9 access control bits, in three groups of three. These correspond to R, W, and X permissions for each of the Owner, Group, and Others. (See "man chmod" for full details.) The RWX bits control the following privileges for ordinary files and directories:

bit	Files	Directories
R	Read (view) file contents.	Read directory contents. Required to get a listing of the directory.
W	Write (change) file	Change directory contents. Required to create or delete files.

	contents.	
X	Execute file contents as a program.	Access detailed directory information. Required to get a long listing, or to access any specific file in the directory. Note that if a user has X but not R permissions on a directory, they can still access specific files, but only if they already know the name of the file they are trying to access.



FILE SYSTEM IMPLEMENTATION

1. File System Structure

- ❖ File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way.
- ❖ A file System must be able to store the file, locate the file and retrieve the file.
- ❖ Most of the Operating Systems use layering approach for every task including file systems.
- ❖ Every layer of the file system is responsible for some activities.

Logical file system

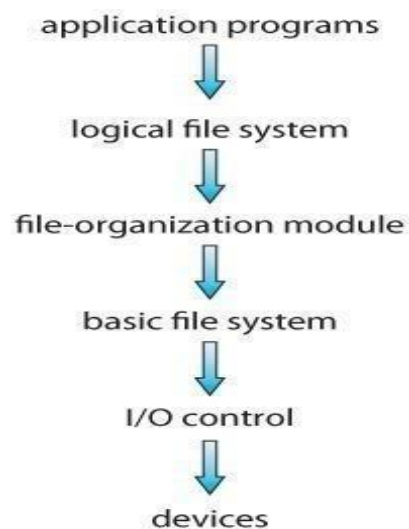
⌘ **Provides** users the view of a contiguous sequence of words, bytes stored somewhere.

⌘ Uses a directory structure, symbolic name

⌘ Provides protection and security

⌘ OS/user interface

⌘ E.g., to create a new file the API provides a call that calls the logical file system



The file organization module

- ⌘ Knows about files and their logical blocks (say 1,..N)
- ⌘ Files are organized in blocks of 32 bytes to 4K bytes
- ⌘ Translates logical blocks into physical
- ⌘ Knows location of file, file allocation type
- ⌘ Includes a free space manager that tracks unallocated blocks

Basic file system

- ⌘ Issues commands to the device driver (layer of software that directly controls disk hardware) to read and write physical blocks on the disk,
- ⌘ Each physical block identified by a disk address (e.g., drive 2, cylinder 34, track 2, sector 11)

IO control

- ⌘ The lowest level in the file system
- ⌘ Consists of device drivers and interrupt handlers to transfer information between the memory and the disk
- ⌘ A device driver translates commands such as “get me block 111” into hardware-specific ISA used by hardware controller. This is accomplished by writing specific bits into IO registers

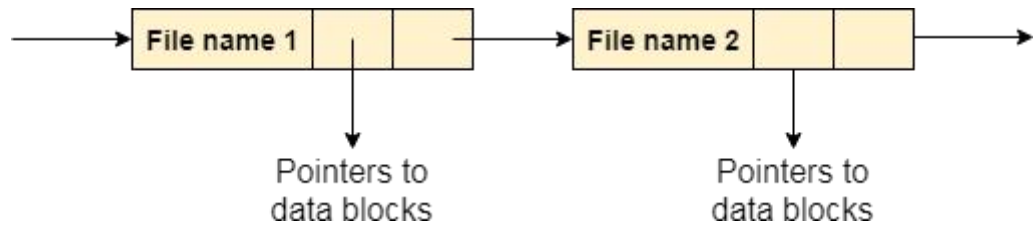
2. Directory Implementation

- Directories need to be fast to search, insert, and delete, with a minimum of wasted disk space.

Linear List

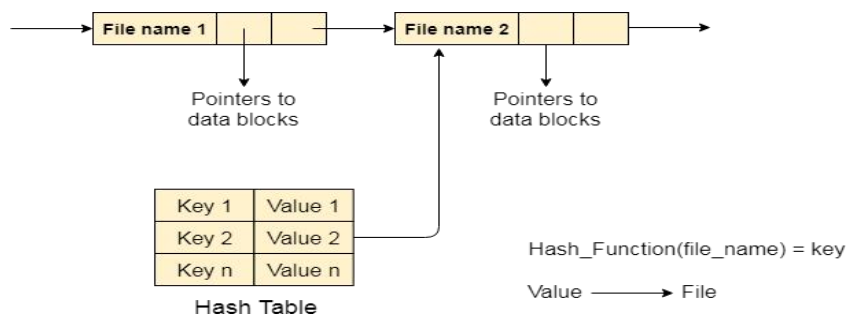
- ❖ A linear list is the simplest and easiest directory structure to set up, but it does have some drawbacks.
- ❖ Finding a file (or verifying one does not already exist upon creation) requires a linear search.
- ❖ Deletions can be done by moving all entries, flagging an entry as deleted, or by moving the last entry into the newly vacant position.

- ❖ Sorting the list makes searches faster, at the expense of more complex insertions and deletions.
- ❖ A linked list makes insertions and deletions into a sorted list easier, with overhead for the links.
- ❖ More complex data structures, such as B-trees, could also be considered.



Hash Table

- ❖ A hash table can also be used to speed up searches.
- ❖ Hash tables are generally implemented in addition to a linear or other structure.
- ❖ A key-value pair for each file in the directory gets generated and stored in the hash table.
- ❖ The key can be determined by applying the hash function on the file name while the key points to the corresponding file stored in the directory.
- ❖ **Searching** → Only hash table entries are checked using the key and if an entry found then the corresponding file will be fetched using the value.

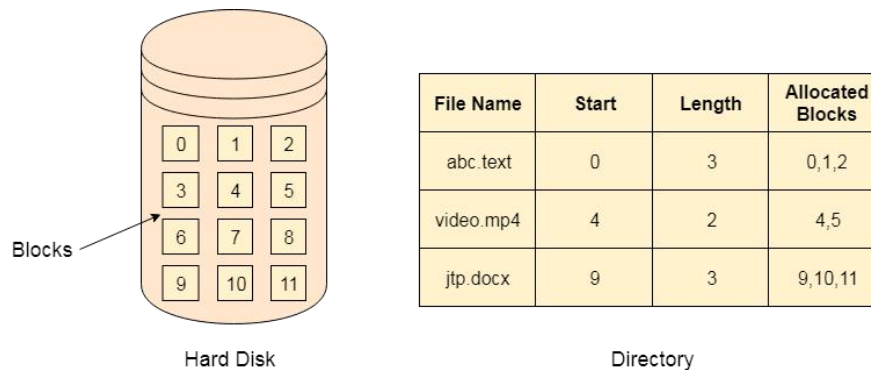


3. Allocation Methods

- ❖ There are various methods which can be used to allocate disk space to the files. Selection of an appropriate allocation method will significantly affect the performance and efficiency of the system.
- ❖ Allocation method provides a way in which the disk will be utilized and the files will be accessed.

Contiguous Allocation

- ❖ If the blocks are allocated to the file in such a way that all the logical blocks of the file get the contiguous physical block in the hard disk then such allocation scheme is known as contiguous allocation.
- ❖ In the image shown below, there are three files in the directory.
- ❖ The starting block and the length of each file are mentioned in the table. We can check in the table that the contiguous blocks are assigned to each file as per its need.



- ❖ All these algorithms suffer from the problem of external fragmentation.
- ❖ As files are allocated and deleted, the free disk space is broken into little pieces. External fragmentation exists whenever free space is broken into chunks.
- ❖ It becomes a problem when the largest contiguous chunk is insufficient for a request; storage is fragmented into a number of holes, none of which is large enough to store the data.
- ❖ This scheme effectively **compacts** all free space into one contiguous space, solving the fragmentation problem.

Advantages

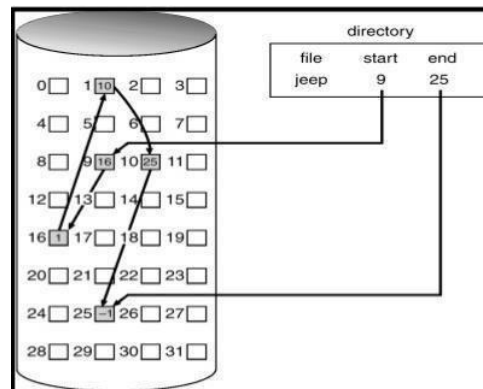
- ❖ It is simple to implement.
- ❖ We will get Excellent read performance.
- ❖ Supports Random Access into files.

Disadvantages

- ❖ The disk will become fragmented.
- ❖ It may be difficult to have a file grow.

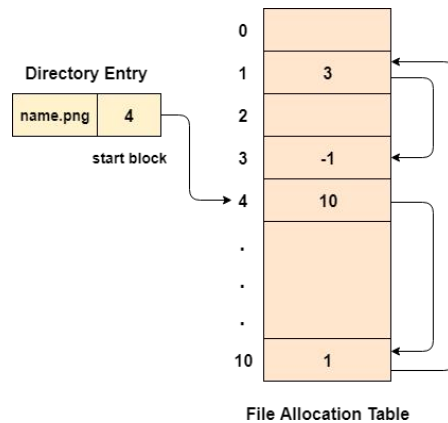
Linked List Allocation

- ❖ Linked List allocation solves all problems of contiguous allocation.
- ❖ In linked list allocation, each file is considered as the linked list of disk blocks.
- ❖ However, the disks blocks allocated to a particular file need not to be contiguous on the disk.
- ❖ Each disk block allocated to a file contains a pointer which points to the next disk block allocated to the same file.
- ❖ For example, a file of five blocks might start at block 9 and continue at block 16, then block 1, then block 10, and finally block 25 (See Figure). Each block contains a pointer to the next block. These pointers are not made available to the user. Thus, if each block is 512 bytes in size, and a disk address (the pointer) requires 4 bytes, then the user sees blocks of 508 bytes.



File Allocation Table

- ❖ The main disadvantage of linked list allocation is that the Random access to a particular block is not provided. In order to access a block, we need to access all its previous blocks.
- ❖ File Allocation Table overcomes this drawback of linked list allocation. In this scheme, a file allocation table is maintained, which gathers all the disk block links. The table has one entry for each disk block and is indexed by block number.
- ❖ File allocation table needs to be cached in order to reduce the number of head seeks. Now the head doesn't need to traverse all the disk blocks in order to access one successive block.



Advantages

- ❖ There is no external fragmentation with linked allocation.
- ❖ Any free block can be utilized in order to satisfy the file block requests.
- ❖ File can continue to grow as long as the free blocks are available.
- ❖ Directory entry will only contain the starting block address.

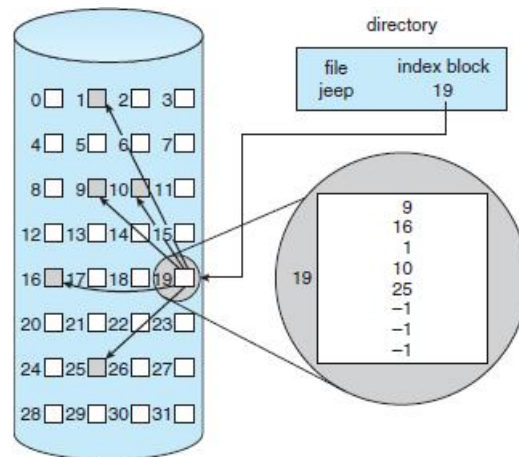
Disadvantages

- ❖ Random Access is not provided.
- ❖ Pointers require some space in the disk blocks.
- ❖ Any of the pointers in the linked list must not be broken otherwise the file will get corrupted.
- ❖ Need to traverse each block.

Indexed Allocation

- ❖ Indexed allocation solves this problem by bringing all the pointers together into one location: **the index block**.
- ❖ Each file has its own index block, which is an array of disk-block addresses.
- ❖ The i th entry in the index block points to the i th block of the file. The directory contains the address of the index block.
- ❖ To find and read the i th block, we use the pointer in the i th index-block entry. This scheme is similar to the paging scheme.
- ❖ When the file is created, all pointers in the index block are set to null.
- ❖ When the i th block is first written, a block is obtained from the free-space manager, and its address is put in the i th index-block entry.

- ❖ Indexed allocation supports direct access, without suffering from external fragmentation,



because any free block on the disk can satisfy a request for more space.

Advantages

1. Supports direct access
2. A bad data block causes the loss of only that block.

Disadvantages

1. A bad index block could cause the loss of entire file.
2. Size of a file depends upon the number of pointers, an index block can hold.
3. Having an index block for a small file is totally wastage.
4. More pointer overhead

4. Free Space Management

Since disk space is limited, we need to reuse the space from deleted files for new files, if possible. To keep track of free disk space, the system maintains a free-space list. The free-space list records all free disk blocks – those not allocated to some file or directory.

To create a file, we search the free-space list for the required amount of space, and allocate that space to the new file. This space is then removed from the free-space list. When a file is deleted, its disk space is added to the free-space list.

1. Bit Vector

The free-space list is implemented as a bit map or bit vector. Each block is represented by 1 bit.

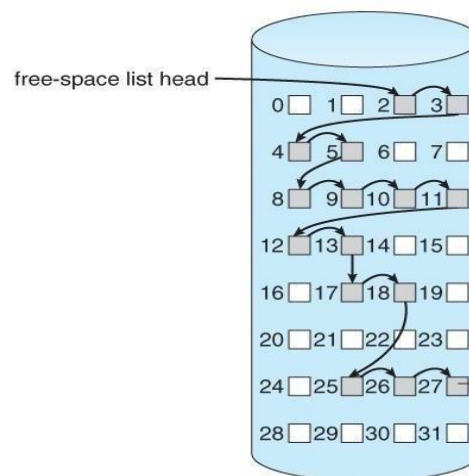
If the block is free, the bit is 1; if the block is allocated, the bit is 0. For example, Consider a disk where block 2,3,4,5,8,9,10,11,12,13,17,18,25,26 and 27 are free, and the rest of the block are allocated. The free space bit map would be 001111001111110001100000011100000 ...

The main advantage of this approach is its relatively simplicity and efficiency in finding the first free block, or n consecutive free blocks on the disk.

2. Linked List

Another approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory. This first block contains a pointer to the next free disk block, and so on.

In our example, we would keep a pointer to block 2, as the first free block. Block 2 would contain a pointer to block 3, which would point to block 4, which would point to block 5, which would point to block 8, and so on. However, this scheme is not efficient; to traverse the list, we must read each block, which requires substantial I/O time. The FAT method incorporates free-block accounting data structure. No separate method is needed.



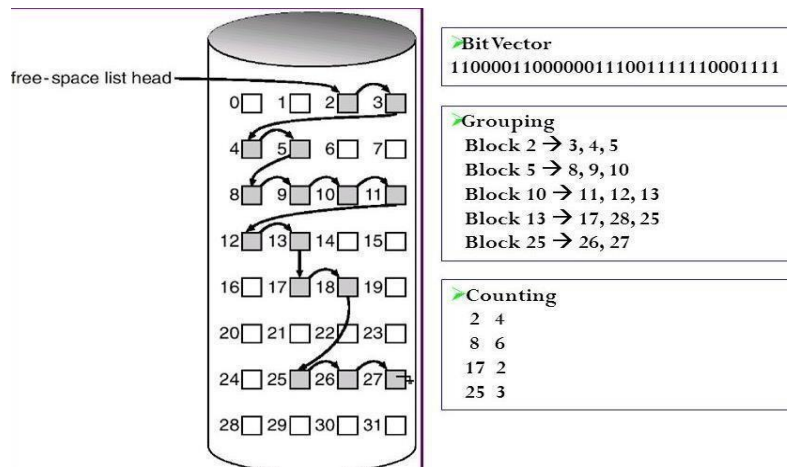
3. Grouping

A modification of the free-list approach is to store the addresses of n free blocks in the first free block. The first n-1 of these blocks are actually free. The last block contains the addresses of another n free blocks, and so on. The importance of this

implementation is that the addresses of a large number of free blocks can be found quickly.

4. Counting

We can keep the address of the first free block and the number n of free contiguous blocks that follow the first block. Each entry in the free-space list then consists of a disk address and a count. Although each entry requires more space than would a simple disk address, the overall list will be shorter, as long as the count is generally greater than 1.



5. Efficiency and Performance

Efficiency

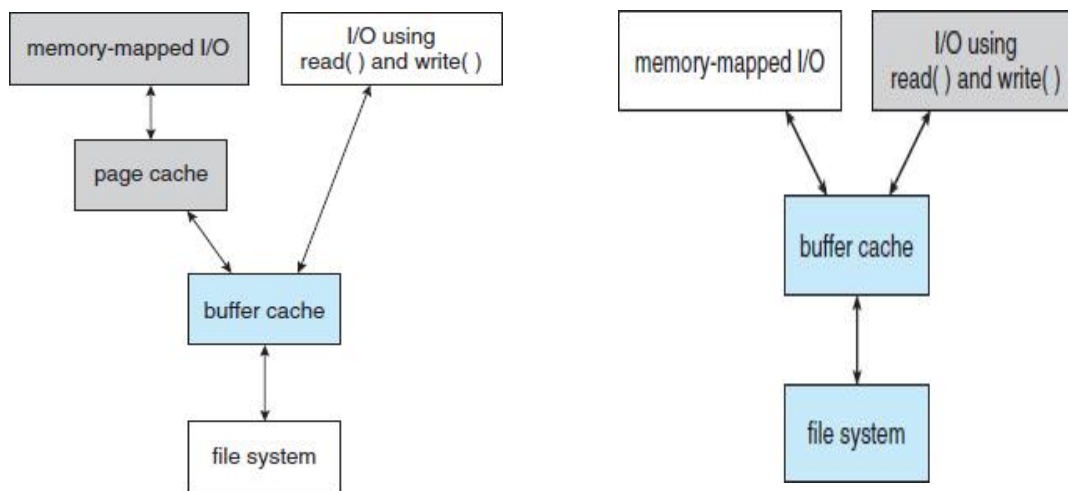
- ❖ The efficient use of disk space depends heavily on the disk-allocation and directory algorithms in use.
- ❖ Let's reconsider the clustering scheme, which improves file-seek and file-transfer performance at the cost of internal fragmentation. To reduce this fragmentation, BSD UNIX varies the cluster size as a file grows. Large clusters are used where they can be filled, and small clusters are used for small files and the last cluster of a file. This
- ❖ The types of data normally kept in a file's directory (or inode) entry also require consideration. Commonly, a "last write date" is recorded to supply information to the user

and to determine whether the file needs to be backed up. Some systems also keep a “last access date,” so that a user can determine when the file was last read.

- ❖ The result of keeping this information is that, whenever the file is read, a field in the directory structure must be written to. That means the block must be read into memory, a section changed, and the block written back out to disk, because operations on disks occur only in block (or cluster) chunks. So any time a file is opened for reading, its directory entry must be read and written as well.
- ❖ Generally, every data item associated with a file needs to be considered for its effect on efficiency and performance.

Performance

- ❖ Some systems maintain a separate section of main memory for a **buffer cache**, where blocks are kept under the assumption that they will be used again shortly. Other systems cache file data using a **page cache**.
- ❖ The **page cache** uses virtual memory techniques to cache file data as pages rather than as file-system-oriented blocks.
- ❖ Caching file data using virtual addresses is far more efficient than caching through physical disk blocks, as accesses interface with virtual memory rather than the filesystem.
- ❖ Several systems—including Solaris, Linux, and Windows —use page caching to cache both process pages and file data. This is known as **unified virtual memory**.



- The two alternatives for opening and accessing a file. One approach is to use memory-mapping the second is to use the standard system calls **read()** and **write()**.
- Here, the read() and write() system calls go through the buffer cache.
- The memory-mapping call, however, requires using two caches— the **page cache** and the **buffer cache**.
- A memory mapping proceeds by reading in disk blocks from the file system and storing them in the buffer cache. Because the virtual memory system does not interface with the buffer cache, the contents of the file in the buffer cache must be copied into the page cache. This situation, known as **double caching**, requires caching file-system data twice.

6. Recovery

Consistency Checking

- ❖ The storing of certain data structures (e.g. directories and inodes) in memory and the caching of disk operations can speed up performance, but what happens in the result of a system crash? All volatile memory structures are lost, and the information stored on the hard drive may be left in an inconsistent state.
- ❖ A Consistency Checker (fsck in UNIX, chkdsk or scandisk in Windows) is often run at boot time or mount time, particularly if a filesystem was not closed down properly. Some of the problems that these tools look for include:
 - Disk blocks allocated to files and also listed on the free list.
 - Disk blocks neither allocated to files nor on the free list.
 - Disk blocks allocated to more than one file.
 - The number of disk blocks allocated to a file inconsistent with the file's stated size.
 - Properly allocated files / inodes which do not appear in any directory entry.
 - Link counts for an inode not matching the number of references to that inode in the directory structure.
 - Two or more identical file names in the same directory.
 - Illegally linked directories, e.g. cyclical relationships where those are not allowed, or files/directories that are not accessible from the root of the directory tree.
 - Consistency checkers will often collect questionable disk blocks into new files with names such as chk00001.dat. These files may contain valuable information that would otherwise be lost, but in most cases they can be safely deleted, (returning those disk blocks to the free list.)

UNIX caches directory information for reads, but any changes that affect space allocation or metadata changes are written synchronously, before any of the corresponding data blocks are written to.

Log-Structured File Systems

➤ Log-based transaction-oriented (a.k.a. journaling) filesystems borrow techniques developed for databases, guaranteeing that any given transaction either completes successfully or can be rolled back to a safe state before the transaction commenced:

- ❖ All metadata changes are written sequentially to a log.
- ❖ A set of changes for performing a specific task (e.g. moving a file) is a transaction.
- ❖ As changes are written to the log they are said to be committed, allowing the system to return to its work.
- ❖ In the meantime, the changes from the log are carried out on the actual file system, and a pointer keeps track of which changes in the log have been completed and which have not yet been completed.
- ❖ When all changes corresponding to a particular transaction have been completed, that transaction can be safely removed from the log.
- ❖ At any given time, the log will contain information pertaining to uncompleted transactions only, e.g. actions that were committed but for which the entire transaction has not yet been completed.
- ❖ From the log, the remaining transactions can be completed, or if the transaction was aborted, then the partially completed changes can be undone.

Backup and Restore

- ❖ In order to recover lost data in the event of a disk crash, it is important to conduct backups regularly.
- ❖ Files should be copied to some removable medium, such as magnetic tapes, CDs, DVDs, or external removable hard drives.
- ❖ A full backup copies every file on a file system. Incremental backups copy only files which have changed since some previous time.

- ❖ A combination of full and incremental backups can offer a compromise between full recoverability, the number and size of backup tapes needed, and the number of tapes that need to be used to do a full restore.

- ❖ A typical backup schedule may then be as follows:

Day 1. Copy to a backup medium all files from the disk. This is called a full backup.

Day 2. Copy to another medium all files changed since day 1. This is an incremental backup.

Day 3. Copy to another medium all files changed since day 2.

Day N. Copy to another medium all files changed since day N-1. Then go back to day 1.